

Optimization Techniques, Assignment 1

Gaurav Choudhury · Roll No. 2501CS26

Big-M Simplex Method · Transportation Problem (VAM and MODI)

Python 3, standard library only. Screenshots: carbon.now.sh, One Dark theme.

1 Big-M simplex method

Code: assignment/big_m_simplex.py

big_m_simplex.py, lines 1-49

```
big_m_simplex.py

1 """big-M simplex. M kept as two fraction rows zM and zc"""
2
3 from fractions import Fraction
4
5
6 class BigMSimplex:
7     def __init__(self, c, A, relations, b, sense="min"):
8         self.n_orig = len(c)
9         self.A = [[Fraction(v) for v in row] for row in A]
10        self.relations = list(relations)
11        self.b = [Fraction(v) for v in b]
12        self.sense = sense
13        self.c = [Fraction(v) for v in c]
14        if sense == "max": # max c*x is just min -c*x
15            self.c = [-v for v in self.c]
16
17    def _build(self):
18        m = len(self.A)
19        slack_col = [None] * m
20        surp_col = [None] * m
21        art_col = [None] * m
22
23        n = self.n_orig
24        for i, rel in enumerate(self.relations):
25            if rel == "<=":
26                slack_col[i] = n
27                n += 1
28            elif rel == ">=":
29                surp_col[i] = n
30                art_col[i] = n + 1
31                n += 2
32            else: # "="
33                art_col[i] = n
34                n += 1
35
36        names = [f"x{j + 1}" for j in range(self.n_orig)]
37        basic = [None] * m
38        for i, rel in enumerate(self.relations):
39            if rel == "<=":
40                names.append(f"s{i + 1}")
41                basic[i] = slack_col[i]
42            elif rel == ">=":
43                names.append(f"e{i + 1}")
44                names.append(f"a{i + 1}")
45                basic[i] = art_col[i]
46            else:
47                names.append(f"a{i + 1}")
48                basic[i] = art_col[i]
49
```

```

big_m_simplex.py
50     rows = [[Fraction(0)] * (n + 1) for _ in range(m)]
51     for i in range(m):
52         for j in range(self.n_orig):
53             rows[i][j] = self.A[i][j]
54         if slack_col[i] is not None:
55             rows[i][slack_col[i]] = Fraction(1)
56         if surp_col[i] is not None:
57             rows[i][surp_col[i]] = Fraction(-1)
58         if art_col[i] is not None:
59             rows[i][art_col[i]] = Fraction(1)
60         rows[i][-1] = self.b[i]
61
62     # min c*x + M*(artificals)
63     zc = [Fraction(0)] * (n + 1)
64     zM = [Fraction(0)] * (n + 1)
65     for j in range(self.n_orig):
66         zc[j] = self.c[j]
67     for i in range(m):
68         if art_col[i] is not None:
69             zM[art_col[i]] = Fraction(1)
70
71     # artificals start basic so clear em from the z row
72     for i in range(m):
73         if art_col[i] is not None:
74             self._eliminate(rows[i], zM, zc, art_col[i])
75
76     return rows, zM, zc, basic, names
77
78     def solve(self, verbose=False):
79         rows, zM, zc, basic, names = self._build()
80         m = len(self.A)
81         n = len(names)
82
83         if verbose:
84             self._show(rows, zM, zc, names, basic)
85
86         iterations = 0
87         while True:
88             # most negative enters. M beats any constant
89             entering = None
90             best = (Fraction(0), Fraction(0))
91             for j in range(n):
92                 if (zM[j], zc[j]) < best:
93                     best, entering = (zM[j], zc[j]), j
94             if entering is None:
95                 break
96
97             leaving = None
98             ratio = None

```

```

big_m_simplex.py

99     for i in range(m):
100         a = rows[i][entering]
101         if a > 0:
102             r = rows[i][-1] / a
103             # on a tie kick the artifical out
104             if (
105                 leaving is None
106                 or r < ratio
107                 or (r == ratio and names[basic[i]].startswith("a"))
108             ):
109                 leaving, ratio = i, r
110     if leaving is None:
111         raise ArithmeticError(
112             "LP unbounded: no positive pivot in entering column."
113         )
114
115     if verbose:
116         print(
117             f"[iter {iterations}] enter {names[entering]} "
118             f"leave {names[basic[leaving]]}"
119         )
120     self._pivot(rows, zM, zc, leaving, entering)
121     basic[leaving] = entering
122     iterations += 1
123     if verbose:
124         self._show(rows, zM, zc, names, basic)
125
126     x = [Fraction(0)] * self.n_orig
127     for i in range(m):
128         if basic[i] < self.n_orig:
129             x[basic[i]] = rows[i][-1]
130
131     # artifical still basic and nonzero = infeasible
132     feasible = all(
133         not (names[basic[i]].startswith("a") and rows[i][-1] != 0) for i in range(m)
134     )
135
136     # z row holds -Z. max got negated in __init__ so flip back
137     z = zc[-1] if self.sense == "max" else -zc[-1]
138
139     return {"x": x, "z": z, "feasible": feasible, "iterations": iterations}
140
141     @staticmethod
142     def _eliminate(pivot_row, zM, zc, col): # pivot_row has to be M free
143         for zrow in (zM, zc):
144             f = zrow[col]
145             if f:
146                 for j in range(len(zrow)):
147                     zrow[j] -= f * pivot_row[j]

```

```

big_m_simplex.py

148
149 @classmethod
150 def _pivot(cls, rows, zM, zc, leaving, entering):
151     p = rows[leaving][entering]
152     rows[leaving] = [v / p for v in rows[leaving]]
153     for i, row in enumerate(rows):
154         if i != leaving and row[entering]:
155             f = row[entering]
156             rows[i] = [v - f * w for v, w in zip(row, rows[leaving])]
157     cls._eliminate(rows[leaving], zM, zc, entering)
158
159 @staticmethod
160 def _show(rows, zM, zc, names, basic):
161     def cell(mm, cc):
162         if not mm:
163             return str(cc)
164         return f"{mm}M{cc:+}" if cc else f"{mm}M"
165
166     print(" ".join(f"{nm:>7}" for nm in names) + " RHS")
167     print(" z| " + " ".join(f"{cell(a, b):>7}" for a, b in zip(zM, zc)))
168     for i, row in enumerate(rows):
169         print(f"{names[basic[i]]:>3}| " + " ".join(f"{str(v):>7}" for v in row))
170     print()
171
172
173 if __name__ == "__main__":
174     # case study diet blend. min 4x1+x2 st 3x1+x2=3 and 4x1+3x2 ≥ 6 and x1+2x2 ≤ 4
175     c = [4, 1]
176     A = [[3, 1], [4, 3], [1, 2]]
177     relations = ["=", "≥", "≤"]
178     b = [3, 6, 4]
179
180     solver = BigMSimplex(c, A, relations, b, sense="min")
181     res = solver.solve(verbose=True)
182
183     print("=" * 46)
184     if not res["feasible"]:
185         print("Infeasible: an artificial variable could not be driven to 0.")
186     else:
187         for k in range(len(c)):
188             print(f" x{k + 1} = {res['x'][k]} ( {float(res['x'][k]):.6f} )")
189         print(f" Z = {res['z']} ( {float(res['z']):.6f} )")
190         print(f" simplex iterations = {res['iterations']}")
191         assert (res["x"][0], res["x"][1], res["z"]) == (
192             Fraction(2, 5),
193             Fraction(9, 5),
194             Fraction(17, 5),
195         ), "known optimum changed"
196         print(" self-check = OK")

```

Output: python3 assignment/big_m_simplex.py

outputs/big_m_output.txt



terminal - big_m_output.txt

```
$ python3 big_m_simplex.py
```

	x1	x2	a1	e2	a2	s3	RHS
z	-7M+4	-4M+1	0	1M	0	0	-9M
a1	3	1	1	0	0	0	3
a2	4	3	0	-1	1	0	6
s3	1	2	0	0	0	1	4

[iter 0] enter x1 leave a1

	x1	x2	a1	e2	a2	s3	RHS
z	0	$-5/3M-1/3$	$7/3M-4/3$	1M	0	0	$-2M-4$
x1	1	1/3	1/3	0	0	0	1
a2	0	5/3	-4/3	-1	1	0	2
s3	0	5/3	-1/3	0	0	1	3

[iter 1] enter x2 leave a2

	x1	x2	a1	e2	a2	s3	RHS
z	0	0	$1M-8/5$	-1/5	$1M+1/5$	0	-18/5
x1	1	0	3/5	1/5	-1/5	0	3/5
x2	0	1	-4/5	-3/5	3/5	0	6/5
s3	0	0	1	1	-1	1	1

[iter 2] enter e2 leave s3

	x1	x2	a1	e2	a2	s3	RHS
z	0	0	$1M-7/5$	0	1M	1/5	-17/5
x1	1	0	2/5	0	0	-1/5	2/5
x2	0	1	-1/5	0	0	3/5	9/5
e2	0	0	1	1	-1	1	1

=====

```
x1 = 2/5    ( 0.400000 )
x2 = 9/5    ( 1.800000 )
Z   = 17/5   ( 3.400000 )
simplex iterations = 3
self-check        = OK
```

2 Vogel's Approximation Method (VAM)

Code: assignment/transportation_vam_modi.py

transportation_vam_modi.py, lines 1-42

```
transportation_vam_modi.py

1 """transportation problem. VAM gets the first BFS then MODI improves it"""
2
3 import sys
4 from fractions import Fraction
5
6
7 class Transportation:
8     def __init__(self, supply, demand, cost):
9         self.supply = [Fraction(v) for v in supply]
10        self.demand = [Fraction(v) for v in demand]
11        self.cost = [[Fraction(v) for v in row] for row in cost]
12        self._balance()
13        self.m = len(self.cost)
14        self.n = len(self.cost[0])
15
16    def _balance(self): # zero cost dummy row/col
17        s, d = sum(self.supply), sum(self.demand)
18        if s > d:
19            self.demand.append(s - d)
20            for row in self.cost:
21                row.append(Fraction(0))
22        elif d > s:
23            self.supply.append(d - s)
24            self.cost.append([Fraction(0)] * len(self.cost[0]))
25
26    def vam(self):
27        m, n = self.m, self.n
28        supply = list(self.supply)
29        demand = list(self.demand)
30        alloc = [[Fraction(0)] * n for _ in range(m)]
31        basis = set()
32
33        active_rows = set(range(m))
34        active_cols = set(range(n))
35
36        # only one cell left so no second cheapest. just use the cost
37        def row_penalty(r):
38            costs = sorted(self.cost[r][c] for c in active_cols)
39            return costs[1] - costs[0] if len(costs) > 1 else costs[0]
40
41        def col_penalty(c):
42            costs = sorted(self.cost[r][c] for r in active_rows)
```

```

transportation_vam_modi.py

43     return costs[1] - costs[0] if len(costs) > 1 else costs[0]
44
45     while active_rows and active_cols:
46         best = None
47         for r in active_rows:
48             p = row_penalty(r)
49             if best is None or p > best[0]:
50                 best = (p, "row", r)
51         for c in active_cols:
52             p = col_penalty(c)
53             if best is None or p > best[0]:
54                 best = (p, "col", c)
55
56         _, kind, idx = best
57
58         if kind == "row":
59             r = idx
60             c = min(active_cols, key=lambda cc: self.cost[r][cc])
61         else:
62             c = idx
63             r = min(active_rows, key=lambda rr: self.cost[rr][c])
64
65         x = min(supply[r], demand[c])
66         alloc[r][c] = x
67         basis.add((r, c))
68         supply[r] -= x
69         demand[c] -= x
70         if supply[r] == 0:
71             active_rows.discard(r)
72         if demand[c] == 0:
73             active_cols.discard(c)
74
75         self._repair_degenerate(alloc, basis)
76         return alloc, basis
77
78     def _repair_degenerate(self, alloc, basis):
79         # cell cant close a loop or the basis isnt a tree and modi stalls
80         m, n = self.m, self.n
81         while len(basis) < m + n - 1:
82             target = None
83             for r in range(m):
84                 for c in range(n):

```

```

transportation_vam_modi.py

85         if (r, c) in basis or alloc[r][c] != 0:
86             continue
87         try:
88             self._find_cycle((r, c), basis)
89         except RuntimeError:
90             target = (r, c)          # no loop so its independant
91             break
92         if target:
93             break
94         if target is None:
95             break
96         basis.add(target)
97
98     def modi(self, alloc, basis, verbose=True):
99         m, n = self.m, self.n
100         steps = 0
101
102         while True:
103             #  $u_i + v_j = c_{ij}$  over the basis.  $u_0 = 0$ . zeros below are a guard
104             u = [None] * m
105             v = [None] * n
106             u[0] = Fraction(0)
107             changed = True
108             while changed:
109                 changed = False
110                 for (i, j) in basis:
111                     if u[i] is not None and v[j] is None:
112                         v[j] = self.cost[i][j] - u[i]; changed = True
113                     elif v[j] is not None and u[i] is None:
114                         u[i] = self.cost[i][j] - v[j]; changed = True
115             for i in range(m):
116                 if u[i] is None:
117                     u[i] = Fraction(0)
118             for j in range(n):
119                 if v[j] is None:
120                     v[j] = Fraction(0)
121
122             entering = None
123             worst = Fraction(0)
124             for r in range(m):
125                 for c in range(n):
126                     if (r, c) in basis:

```

```

transportation_vam_modi.py

127         continue
128         d = self.cost[r][c] - u[r] - v[c]
129         if d < worst:
130             worst, entering = d, (r, c)
131
132     if entering is None:                # all d ≥ 0 so were done
133         break
134
135     # push theta round the loop. even gain odd give up
136     loop = self._find_cycle(entering, basis)
137     theta = None
138     for k in range(1, len(loop), 2):
139         q = alloc[loop[k][0]][loop[k][1]]
140         if theta is None or q < theta:
141             theta = q
142     for k, (r, c) in enumerate(loop):
143         if k % 2 == 0:
144             alloc[r][c] += theta
145         else:
146             alloc[r][c] -= theta
147     basis.add(entering)
148     # only one leaves even if the pivot empties a few
149     for k in range(1, len(loop), 2):
150         r, c = loop[k]
151         if alloc[r][c] == 0:
152             basis.discard((r, c))
153         break
154     steps += 1
155     if verbose:
156         print(f" MODI iter {steps}: enter ({entering[0]+1},{entering[1]+1})"
157               f" theta = {theta}")
158
159     return alloc, basis, steps
160
161 def _find_cycle(self, start, basis):
162     # row close only. vertical lets in odd loops that break the totals
163     path = [start]
164
165     def dfs(cell, came_horizontal):
166         r, c = cell
167         if came_horizontal:                # go vertical now
168             for i in range(self.m):

```

```

transportation_vam_modi.py

169         nxt = (i, c)
170         if nxt in basis and nxt not in path:
171             path.append(nxt)
172             if dfs(nxt, False):
173                 return True
174             path.pop()
175         else:                                     # horizontal now
176             for j in range(self.n):
177                 nxt = (r, j)
178                 if nxt == start and len(path) > 2:
179                     return True
180                 if nxt in basis and nxt not in path:
181                     path.append(nxt)
182                     if dfs(nxt, True):
183                         return True
184                     path.pop()
185             return False
186
187         if dfs(start, True):
188             return path
189         raise RuntimeError("no closed loop found for entering cell")
190
191     def total_cost(self, alloc):
192         total = Fraction(0)
193         for r in range(self.m):
194             for c in range(self.n):
195                 total += self.cost[r][c] * alloc[r][c]
196         return total
197
198     def show(self, alloc, basis=None):
199         print(" allocation matrix (rows = sources, cols = destinations):")
200         for r in range(self.m):
201             cells = []
202             for c in range(self.n):
203                 tag = "*" if basis and (r, c) in basis else " "
204                 cells.append(f"{alloc[r][c]}{tag}")
205             print("    " + " ".join(f"{x:>6}" for x in cells))
206
207
208 if __name__ == "__main__":
209     # case study cement distribution. 3 plants to 4 warehouses
210     demand = [10, 40, 20, 30]

```

```

211     supply = [20, 30, 50]
212     cost = [[8, 6, 10, 9],      # plant S1
213             [9, 12, 13, 7],    # plant S2
214             [14, 9, 16, 5]]    # plant S3
215
216     tp = Transportation(supply, demand, cost)
217     only_vam = "--vam-only" in sys.argv
218
219     print("Transportation problem: 3 sources x 4 destinations")
220     print("  supply  :", supply)
221     print("  demand  :", demand)
222     print("  costs   :")
223     for row in cost:
224         print("          ", row)
225
226     print("\n[Phase 1] Vogel's Approximation Method")
227     alloc, basis = tp.vam()
228     tp.show(alloc, basis)
229     vam_cost = tp.total_cost(alloc)
230     print(f"  VAM initial BFS cost = {vam_cost} ( {float(vam_cost):.2f} )")
231
232     if only_vam:
233         sys.exit(0)
234
235     print("\n[Phase 2] MODI optimality test / improvement")
236     alloc, basis, iters = tp.modi(alloc, basis)
237     tp.show(alloc, basis)
238     opt = tp.total_cost(alloc)
239     print(f"  MODI iterations      = {iters}")
240     print(f"  optimal total cost    = {opt} ( {float(opt):.2f} )")
241
242     row_sum = [sum(alloc[r][c] for c in range(tp.n)) for r in range(tp.m)]
243     col_sum = [sum(alloc[r][c] for r in range(tp.m)) for c in range(tp.n)]
244     lst = lambda xs: "[" + ", ".join(str(v) for v in xs) + "]"
245     print("\n  check: row sums  =", lst(row_sum), " (supply:", supply, ")")
246     print("  check: col sums  =", lst(col_sum), " (demand:", demand, ")")
247     print("  basis cells      =", len(basis), " (m+n-1 =", tp.m + tp.n - 1, ")")
248
249     assert row_sum == tp.supply and col_sum == tp.demand, "allocation infeasible"
250     assert all(alloc[r][c] >= 0 for r in range(tp.m) for c in range(tp.n))
251     assert opt == 800, f"known optimum for this case study is 800, got {opt}"
252     print("  self-check      = OK")

```

Output: python3 assignment/transportation_vam_modi.py -vam-only

outputs/vam_output.txt

```
terminal - vam_output.txt

$ python3 transportation_vam_modi.py --vam-only

Transportation problem: 3 sources x 4 destinations
supply   : [20, 30, 50]
demand   : [10, 40, 20, 30]
costs    :
           [8, 6, 10, 9]
           [9, 12, 13, 7]
           [14, 9, 16, 5]

[Phase 1] Vogel's Approximation Method
allocation matrix (rows = sources, cols = destinations):
    0*    20*    0    0
   10*    0   20*    0
    0   20*    0   30*
VAM initial BFS cost = 800 ( 800.00 )
```

3 MODI optimality test / improvement

Code (excerpt): `modi` and `_find_cycle`

from `assignment/transportation_vam_modi.py`

`transportation_vam_modi.py`, lines 98-131

```
transportation_vam_modi.py - modi

98  def modi(self, alloc, basis, verbose=True):
99      m, n = self.m, self.n
100      steps = 0
101
102      while True:
103          #  $u_i + v_j = c_{ij}$  over the basis.  $u_0 = 0$ . zeros below are a guard
104          u = [None] * m
105          v = [None] * n
106          u[0] = Fraction(0)
107          changed = True
108          while changed:
109              changed = False
110              for (i, j) in basis:
111                  if u[i] is not None and v[j] is None:
112                      v[j] = self.cost[i][j] - u[i]; changed = True
113                  elif v[j] is not None and u[i] is None:
114                      u[i] = self.cost[i][j] - v[j]; changed = True
115              for i in range(m):
116                  if u[i] is None:
117                      u[i] = Fraction(0)
118              for j in range(n):
119                  if v[j] is None:
120                      v[j] = Fraction(0)
121
122          entering = None
123          worst = Fraction(0)
124          for r in range(m):
125              for c in range(n):
126                  if (r, c) in basis:
127                      continue
128                  d = self.cost[r][c] - u[r] - v[c]
129                  if d < worst:
130                      worst, entering = d, (r, c)
131
```

```

transportation_vam_modi.py - modi (contd.)
132     if entering is None:                # all d ≥ 0 so were done
133         break
134
135     # push theta round the loop. even gain odd give up
136     loop = self._find_cycle(entering, basis)
137     theta = None
138     for k in range(1, len(loop), 2):
139         q = alloc[loop[k][0]][loop[k][1]]
140         if theta is None or q < theta:
141             theta = q
142     for k, (r, c) in enumerate(loop):
143         if k % 2 == 0:
144             alloc[r][c] += theta
145         else:
146             alloc[r][c] -= theta
147     basis.add(entering)
148     # only one leaves even if the pivot empties a few
149     for k in range(1, len(loop), 2):
150         r, c = loop[k]
151         if alloc[r][c] == 0:
152             basis.discard((r, c))
153         break
154     steps += 1
155     if verbose:
156         print(f" MODI iter {steps}: enter ({entering[0]+1},{entering[1]+1})"
157               f" theta = {theta}")
158
159     return alloc, basis, steps
160

```

```

transportation_vam_modi.py - _find_cycle

161 def _find_cycle(self, start, basis):
162     # row close only. vertical lets in odd loops that break the totals
163     path = [start]
164
165     def dfs(cell, came_horizontal):
166         r, c = cell
167         if came_horizontal:                # go vertical now
168             for i in range(self.m):
169                 nxt = (i, c)
170                 if nxt in basis and nxt not in path:
171                     path.append(nxt)
172                     if dfs(nxt, False):
173                         return True
174                     path.pop()
175             else:                          # horizontal now
176                 for j in range(self.n):
177                     nxt = (r, j)
178                     if nxt == start and len(path) > 2:
179                         return True
180                     if nxt in basis and nxt not in path:
181                         path.append(nxt)
182                         if dfs(nxt, True):
183                             return True
184                         path.pop()
185             return False
186
187     if dfs(start, True):
188         return path
189     raise RuntimeError("no closed loop found for entering cell")
190

```

Output: python3 assignment/transportation_vam_modi.py

outputs/vam_modi_output.txt

```
terminal - vam_modi_output.txt

$ python3 transportation_vam_modi.py

Transportation problem: 3 sources x 4 destinations
supply  : [20, 30, 50]
demand  : [10, 40, 20, 30]
costs   :
          [8, 6, 10, 9]
          [9, 12, 13, 7]
          [14, 9, 16, 5]

[Phase 1] Vogel's Approximation Method
allocation matrix (rows = sources, cols = destinations):
    0*    20*    0    0
    10*    0    20*    0
    0    20*    0    30*
VAM initial BFS cost = 800 ( 800.00 )

[Phase 2] MODI optimality test / improvement
MODI iter 1: enter (1,3)  theta = 0
allocation matrix (rows = sources, cols = destinations):
    0    20*    0*    0
    10*    0    20*    0
    0    20*    0    30*
MODI iterations      = 1
optimal total cost   = 800 ( 800.00 )

check: row sums = [20, 30, 50] (supply: [20, 30, 50] )
check: col sums = [10, 40, 20, 30] (demand: [10, 40, 20, 30] )
basis cells      = 6 (m+n-1 = 6 )
self-check       = OK
```